Document 10

Golden Test Pack Specification

1. Purpose of This Document

This document defines the required golden test pack for Trinity Engine v1.

Its purpose is to specify:

- the role of the golden test pack
- the categories of required test cases
- the structure of each golden test case
- the matching rules for expected outputs
- the treatment of invalid and unsupported input cases
- the regression obligations tied to canon changes

This document defines how the canon is verified in practice.

It does not replace the canon documents themselves.
The golden test pack verifies the canon. It does not override it.

---

2. General Test Pack Principles

2.1 Canon Verification Rule

The golden test pack exists to verify that the implementation matches the final canon package.

If a test fixture conflicts with the canon:

- the canon governs
- the fixture must be corrected
- the inconsistency must be recorded and resolved explicitly

2.2 Deterministic Repeatability Rule

Every golden test case must be repeatable.

Given:

- the same normalized input
- the same canon version

- the same source stack version
- the same input schema version
- the same mapping version
- the same engine version where behavior depends on implementation

the test must produce the same canonical output or the same canonical error.

2.3 No Approximate Pass Rule

Golden tests must not pass on:

- close enough output
- fuzzy equivalence
- visually similar structure
- reordered collections
- alternate formatting
- equivalent but differently represented identifiers

Golden tests are exact unless the canon explicitly defines a comparison tolerance for a specific value type.

2.4 One Source of Expected Truth Rule

Each golden test case must contain exactly one authoritative expected result set for its pinned version context.

No test case may rely on:

- external calculator screenshots
- forum references
- remembered outputs
- undocumented prototype runs

---

3. Test Pack Scope

3.1 Required Test Categories

The golden test pack for Trinity Engine v1 must contain test cases for at minimum:

1. Valid baseline cases
2. Valid edge cases
3. Invalid input cases
4. Incomplete input cases
5. Unsupported input cases
6. Canon conflict simulation cases where applicable
7. Regression sentinel cases for known ambiguity prone boundaries

3.2 Minimum Coverage Rule

The final test pack must cover all three supported domains:

- Astrology
- Human Design
- Gene Keys

It must also cover:

- input validation behavior
- deterministic metadata behavior
- output ordering behavior
- formatting behavior where formatting is canon pinned

---

4. Golden Test Case Structure

4.1 Required Fields Per Test Case

Each golden test case must contain at minimum:

- `case_id`
- `description`
- `status`
- `category`
- `version_context`
- `input`
- `expected_result`

4.2 `case_id`

`case_id` must be unique across the full test pack.

Recommended format:

- `GT-001`
- `GT-002`
- `GT-003`

Or, if categorization is embedded:

- `VALID-001`
- `INVALID-001`
- `EDGE-001`

Only one format should be used consistently.

4.3 `description`

A short human readable description of what the case is testing.

Example:

- `Valid baseline birth case with full Astrology, Human Design, and Gene Keys output`
- `Invalid input because latitude is provided as string`
- `Boundary case where gate interval end must roll into next gate`

4.4 `status`

Allowed values:

- `active`
- `deprecated`

A deprecated case remains in history but is not part of the current required pass set unless explicitly stated.

4.5 `category`

Allowed values:

- `valid_baseline`
- `valid_edge`
- `invalid_input`
- `incomplete_input`
- `unsupported_input`
- `canon_conflict`
- `regression_sentinel`

4.6 `version_context`

The `version_context` object must contain:

- `canon_version`
- `source_stack_version`
- `input_schema_version`
- `mapping_version`

It may also contain:

- `engine_version` if the golden fixture is pinned to a specific implementation release for audit purposes

4.7 `input`

The `input` object must contain the exact canonical input payload used for the case, matching Document 04.

4.8 `expected_result`

The `expected_result` object must contain either:

- the full expected success response
  or
- the full expected error response

No test case may specify only a partial expected result unless the canon explicitly defines that case as a scoped assertion test rather than a full golden case.

---

5. Success Case Matching Rules

5.1 Full Response Matching Rule

For a success case, the implementation result must match the expected result according to the canonical success comparison basis.

Canonical success comparison requires exact agreement with respect to:

- top level fields
- required nested fields
- canonical field names
- canonical identifiers
- ordered collections where order is canon pinned
- numeric formatting where pinned
- datetime formatting where pinned
- metadata fields
- canonical values

Canonical success comparison does not require literal byte for byte JSON serialization unless a later canon revision explicitly pins serializer behavior and object member ordering.

5.2 Ordered Collection Matching Rule

The following ordered outputs must match exactly in order:

- astrology house cusp order
- astrology object order
- Human Design Personality activation order
- Human Design Design activation order
- center order
- any other ordered array pinned in Document 07

5.3 Numeric Matching Rule

Numeric outputs must match according to the canon pinned numeric precision.

For v1:

- longitude values must match at 6 decimal places exactly as emitted in canonical output

5.4 Datetime Matching Rule

Pinned datetime outputs must match exactly in canonical string format.

For v1:

- `design_time_utc` must match exactly in ISO 8601 UTC with trailing `Z` and whole second precision

5.5 Canonical Success Comparison Basis

Success fixture comparison for Trinity Engine v1 must be based on semantic JSON equality against the canonical output contract.

This means:

- all required fields must be present
- no canonical field name may differ
- exact scalar values must match after canonical formatting rules are applied
- ordered arrays must match exactly where canonical order is specified
- object member ordering is not authoritative unless explicitly pinned by later canon revision

A success case must fail if:

- a required field is missing
- an ordered collection is reordered contrary to canon
- a scalar value differs from the canonical expected value

- a canonical identifier differs
- a canon pinned numeric or datetime representation differs

Literal byte for byte JSON equivalence is not required unless explicitly pinned by later canon revision.

---

6. Error Case Matching Rules

6.1 Error Shape Matching Rule

For an error case, the implementation result must match the expected error contract exactly with respect to:

- `status`
- `metadata`
- `error.error_type`
- `error.error_code`

The `error.message` field must be present if required by Document 07, but its wording is not authoritative for canonical fixture comparison unless a later canon revision explicitly pins a canonical message catalog.

6.2 Error Type Matching Rule

The implementation must emit the exact expected canonical `error_type`.

The implementation must also emit the exact expected canonical `error_code` for the specific failure condition.

No substitution with a neighboring error class is allowed.
No substitution with a neighboring error code is allowed.

Allowed `error_type` values are defined in Document 07.

6.3 Error Message Rule

The error message must satisfy the structural and content constraints defined in Document 07.

For Trinity Engine v1, canonical fixture comparison does not require exact equality of `error.message` wording.

Exact message comparison is allowed only if:

- a later canon revision introduces an authoritative message catalog
- the relevant fixture or fixture class explicitly opts into message exactness under that pinned catalog

Until then, exact message matching is not part of canonical pass criteria.

---

7. Required Valid Baseline Cases

7.1 Baseline Full Output Case

At least **1** case must verify:

- accepted normalized input
- successful Astrology output
- successful Human Design output
- successful Gene Keys output
- required metadata
- required input echo
- correct ordering
- correct formatting

This is the main canonical baseline case.

7.2 Minimum Baseline Case Count

The golden test pack must contain at least **3 active valid baseline cases** in total.

These baseline cases must differ meaningfully in:

- date
- time
- timezone
- coordinates
- Human Design structural outcome

The purpose of this minimum is to reduce overfitting to one birth profile.

---

8. Required Valid Edge Cases

8.1 Boundary Sensitive Cases

The test pack must include valid edge cases for at minimum:

- sign boundary behavior
- house boundary behavior, including cusp ownership where practically testable
- gate boundary behavior
- line boundary behavior
- longitude normalization behavior
- Earth opposition derivation
- South Node opposition derivation where applicable
- Design time search behavior near target precision threshold
- Design time final timestamp selection when stop condition 2 is triggered

8.2 Wraparound Cases

The test pack must include valid edge cases where:

- longitude wraps across `0°`
- gate intervals wrap across the mandala boundary
- zodiac interval ownership at `360°` to `0°` is exercised

8.3 Minimum Edge Case Count

The golden test pack must contain at least **5 active valid edge cases** in total.

These cases must collectively exercise both:

- numerical boundary behavior
- structural derivation behavior

They do not need to exhaust every possible edge condition in v1, but they must prove that the implementation is not only correct for ordinary baseline inputs.

---

9. Required Invalid, Incomplete, and Unsupported Cases

9.1 Invalid Input Cases

The test pack must include invalid cases for at minimum:

- malformed `birth_date`
- malformed `birth_time`
- invalid timezone format
- latitude outside range
- longitude outside range
- latitude provided as string
- longitude provided as string

The golden test pack must contain at least **5 active invalid input cases** in total.

9.2 Incomplete Input Cases

The test pack must include incomplete cases for at minimum:

- missing `birth_date`
- missing `birth_time`
- missing `timezone`
- missing `latitude`
- missing `longitude`

The golden test pack must contain at least **5 active incomplete input cases** in total.

9.3 Unsupported Input Cases

The test pack must include unsupported cases where structurally valid input falls outside supported v1 scope.

Examples may include:

- multi person payload if artificially submitted
- unsupported payload shape that passes basic parsing but violates v1 scope
- any future explicitly unsupported mode defined elsewhere in canon

The golden test pack must contain at least **2 active unsupported input cases** in total.

---

10. Canon Conflict and Regression Sentinel Cases

10.1 Canon Conflict Simulation Cases

Where feasible, the test pack should include cases that verify the engine does not silently proceed when canon critical behavior is undefined or inconsistent.

These cases may be more difficult to automate and may be marked as controlled validation cases rather than normal runtime fixtures if needed.

10.2 Regression Sentinel Cases

10.2 Regression Sentinel Cases

The test pack must include regression sentinel cases for previously identified ambiguity prone areas, including at minimum:

- strict numeric type handling
- gate interval boundary ownership
- line interval boundary ownership
- house interval ownership
- Design time derivation mechanics
- Design time final timestamp emission under sub second bracket stop condition
- channel activation deduplication
- center definition logic
- Definition assignment logic
- Type assignment priority
- Authority assignment priority
- rejected old gate anchor assumptions not reappearing implicitly

These cases exist specifically to prevent old ambiguities from re entering the implementation.

10.3 Minimum Regression Sentinel Count

The golden test pack must contain at least **3 active regression sentinel cases** in total.

At least one sentinel case must target:

- input boundary strictness
- mapping boundary behavior
- Human Design structural derivation behavior

---

11. Fixture Storage and Organization Rules

11.1 Fixture Structure

The golden test pack should be organized in a deterministic file structure.

Recommended structure:

- `valid_baseline/`
- `valid_edge/`
- `invalid_input/`
- `incomplete_input/`
- `unsupported_input/`
- `regression_sentinel/`

11.2 One Fixture One Truth Rule

Each fixture should contain:

- canonical input
- pinned version context
- canonical expected result

This may be stored:

- in one file per case
  or
- in a tightly controlled paired structure

But the relationship must be unambiguous.

11.3 No Hidden Fixture Mutation Rule

Golden fixtures must not be updated silently.

Any fixture change that affects expected behavior must be treated as:

- either a canon correction
- or a regression fix
- or an intentional canon change

And must be documented accordingly.

11.4 Fixture Provenance and Approval Metadata

Each active golden fixture must include or be accompanied by deterministic provenance metadata sufficient to identify:

- fixture authoring date
- canonical version context
- originating engine version if applicable
- approval status
- approval basis
- approved by
- review notes reference where applicable

Minimum allowed approval status values:

- draft
- candidate
- approved
- deprecated

A fixture may be used as part of the canonical required pass set only when its approval status is `approved`.

11.5 Initial Fixture Freeze Rule

The first active approved fixture set for Trinity Engine v1 must not be frozen solely on the basis of a single implementation run.

Before any fixture enters the approved frozen baseline set:

- the fixture input must be confirmed as canon compliant
- the fixture output must be generated by a canon compliant implementation
- the expected result must be reviewed against the governing canon documents for boundary and structural compliance
- at least one independent review pass must be completed for ambiguity prone areas
- the fixture must be explicitly marked approved

At minimum, independent review must cover:

- timezone sensitive cases
- sign boundary cases
- house boundary cases
- gate boundary cases
- line boundary cases
- Design time sub second stop condition cases
- error classification edge cases

The purpose of this rule is to prevent the first fixture set from becoming self validating merely because it was generated by the same implementation later judged against it.

---

12. Regression and Change Control

12.1 Required Re Run Rule

Any change that may affect canonical behavior must trigger re execution of the full relevant golden test pack.

12.2 Failure Interpretation Rule

If a previously passing golden case fails after a change, one of the following must be true:

- the implementation is defective
- the expected fixture was wrong and must be corrected explicitly
- the canon changed and versioning must be updated accordingly

12.3 No Equivalent Enough Rule

A changed result may not be accepted merely because it appears:

- numerically close
- logically similar
- visually equivalent
- structurally reasonable

The golden pack exists to reject that kind of hand waving.

---

13. Example Golden Test Case Shape

```
{
 "case_id": "VALID-001",
 "description": "Valid baseline full output case",
 "status": "active",
 "category": "valid_baseline",
 "version_context": {
   "canon_version": "v1.0",
   "source_stack_version": "v1.0",
   "input_schema_version": "v1.0",
   "mapping_version": "v1.0"
 },
 "input": {
   "birth_date": "1990-04-09",
   "birth_time": "18:04",
   "timezone": "Europe/Amsterdam",
   "latitude": 51.9194,
   "longitude": 4.4792
 },
 "expected_result": {
   "status": "success",
   "metadata": {},
   "input_echo": {},
   "astrology": {},
   "human_design": {},
   "gene_keys": {}
 }
}
```


This example illustrates shape only.
Real fixtures must contain the full canonical expected payload.

---

14. Out of Scope Test Behavior

The golden test pack must not rely on:

- screenshots as expected truth
- manual visual comparison
- forum posts
- undocumented prototype output
- tolerance based matching unless canon explicitly allows it
- third party platform outputs as authoritative expected results

Third party references may help discover issues, but they are not golden truth.

---

15. Dependency Notes

This document must remain aligned with:

- Document 04. Input Specification
- Document 05. Calculation Canon
- Document 06. Mapping Canon
- Document 07. Output Specification
- Document 08. Determinism and Versioning Rules

This document cannot be treated as final until:

- the actual first set of golden fixtures is created
- the fixture categories are populated
- the baseline case set is pinned
- error comparison rules are aligned with Document 07
- success comparison semantics are aligned with Document 07
- the first approved fixture set has completed the fixture freeze rule defined in this document

---

Appendix A

Fixture Authoring Template and First Baseline Fixture Scaffold

Purpose

This appendix provides:

- the canonical fixture authoring template for Trinity Engine v1 golden cases
- the scaffold for the first valid baseline fixture

This appendix does not replace the golden test pack specification.
It provides the operational starting point for creating the first real fixture files.

A1. Canonical Fixture Authoring Template


```
{
 "case_id": "VALID-001",
 "description": "Valid baseline full output case",
 "status": "active",
 "category": "valid_baseline",
 "version_context": {
   "canon_version": "v1.0",
   "source_stack_version": "v1.0",
   "input_schema_version": "v1.0",
   "mapping_version": "v1.0"
 },
 "input": {
   "birth_date": "YYYY-MM-DD",
   "birth_time": "HH:MM",
   "timezone": "Area/Location",
   "latitude": 0.000000,
   "longitude": 0.000000
 },
 "expected_result": {
   "status": "success",
   "metadata": {
     "engine_version": "v1.0.0",
     "canon_version": "v1.0",
     "source_stack_version": "v1.0",
     "input_schema_version": "v1.0",
     "mapping_version": "v1.0"
   },
   "input_echo": {
     "birth_date": "YYYY-MM-DD",
     "birth_time": "HH:MM",
     "timezone": "Area/Location",
     "latitude": 0.000000,
     "longitude": 0.000000
   },
   "astrology": {},
   "human_design": {},
   "gene_keys": {}
 }
}
```

Authoring Rule

The placeholder structure above defines the required fixture shape only.

No expected value may be guessed or hand invented.

All expected values must be populated from an actual canon compliant engine run and must then be reviewed against the governing canon before being frozen as an authoritative golden result.

Generation by a canon compliant engine is necessary but not sufficient for approval.

A candidate expected result becomes an approved golden result only after:

- canon compliance review
- boundary case review where relevant
- explicit approval under the fixture freeze rule defined in this document

---

A2. First Baseline Fixture Scaffold

The first baseline fixture for Trinity Engine v1 should use the following normalized input:


```
{
 "case_id": "VALID-001",
 "description": "Valid baseline full output case using normalized Dutch birth input",
 "status": "active",
 "category": "valid_baseline",
 "version_context": {
   "canon_version": "v1.0",
   "source_stack_version": "v1.0",
   "input_schema_version": "v1.0",
   "mapping_version": "v1.0"
 },
 "input": {
   "birth_date": "1990-04-09",
   "birth_time": "18:04",
   "timezone": "Europe/Amsterdam",
   "latitude": 51.919400,
   "longitude": 4.479200
 },
 "expected_result": {
   "status": "success",
   "metadata": {
     "engine_version": "v1.0.0",
     "canon_version": "v1.0",
     "source_stack_version": "v1.0",
```

```
    "input_schema_version": "v1.0",
    "mapping_version": "v1.0"
  },
  "input_echo": {
    "birth_date": "1990-04-09",
    "birth_time": "18:04",
    "timezone": "Europe/Amsterdam",
    "latitude": 51.919400,
    "longitude": 4.479200
  },
  "astrology": "__FILL_FULL_CANON_OUTPUT__",
  "human_design": "__FILL_FULL_CANON_OUTPUT__",
  "gene_keys": "__FILL_FULL_CANON_OUTPUT__"
 }
}
```


Freeze Rule

For `VALID-001`:

- the input block above is fixed
- the `expected_result` must be populated only from the finalized canon compliant engine output
- that populated result must be reviewed against the governing canon and approved according to the fixture freeze rule in this document
- once verified and approved, that result becomes the frozen baseline golden fixture

`VALID-001` must not be treated as final merely because it was first generated successfully.

A3. Population Rule

Appendix A does not complete Document 10 by itself.

Document 10 remains operationally incomplete until:

- `VALID-001` is populated with real expected results
- the remaining minimum fixture counts are populated with actual cases
- the first edge, invalid, incomplete, unsupported, and regression sentinel fixtures are created